EvoRank: LLM-Guided Evolution of Multi-Objective Learning-to-Rank Pipelines

An evolutionary loop that engineers ranking pipelines, and the audit that keeps it honest.

Rayhan Patel[1,†] Shabaz Patel[2,†]

[1]University of Maryland, College Park [2]Independent Researcher [†]equal contribution

GenAIECommerce'26: The Third Workshop on Agentic and Generative AI for E-Commerce, co-located with RecSys 2026, Minneapolis, MN, USA

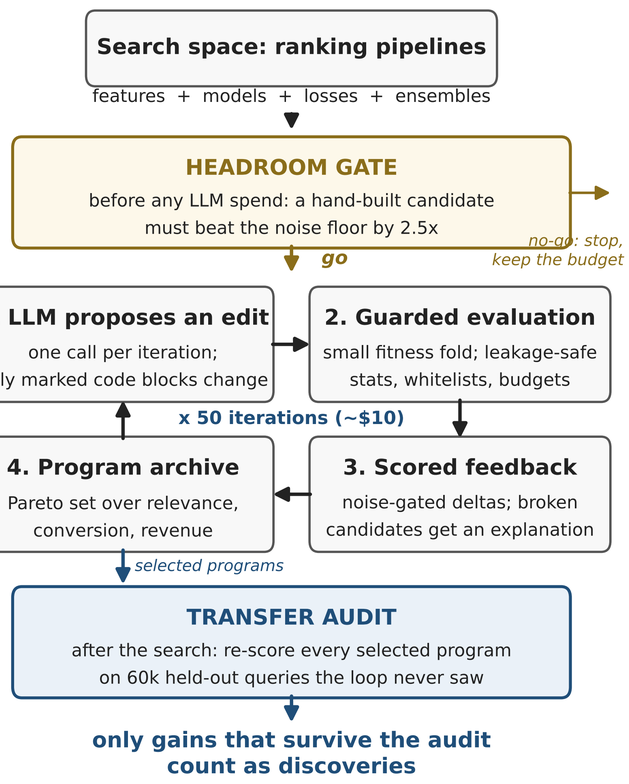
code + all results

The failure nobody reports

Evolutionary loops that pair an LLM with a scored evaluator now design ranking systems end to end: propose a change, measure it, keep the best score. But ranking measurements are noisy, and a selector that always takes the top score will happily select noise, reporting steady progress while discovering nothing that holds up on unseen data. We built such a loop, caught it doing exactly this, measured why, and turned the fix into a procedure.

The claim. Whether the loop will pay off is measurable before any LLM spend. The deciding quantity is the ratio of the effect sizes the search space can offer to the noise of the fitness signal. We package the pre-check as the headroom gate, and we let a transfer audit on held-out queries decide what counts, never the loop's own scores.

One loop, gated and audited



The full audit stack, applied identically to every experiment: paired query bootstrap on every comparison; an equal-budget random-search control; equal Optuna tuning on both sides; the transfer audit; three-objective hypervolume; and component-removal tests on every discovered program.

Task. Expedia hotel search (ICDM 2013): 9.9M impressions, 399k queries, split 70/15/15 by query. Three objectives that genuinely compete, computed from real behavior: relevance (NDCG@10), conversion (booking NDCG@10), and revenue (pooled dollar-weighted capture).

Campaign 1: the loop selects noise

We first let the loop evolve only the training objective, the LambdaMART gradient. On its selection fold it improved steadily. The transfer audit told a different story.

On 60k held-out queries: the best evolved objective beats its own starting program by **+0.0002** NDCG@10 ($p = 0.76$); blind random search at the same budget transfers as well or better ($p = 0.03$); and every method sits below the tuned LambdaMART baseline ($p < 0.001$).

Why, measured: one component's true effect is **0.0005** to **0.008** NDCG@10, while the fold's measurement noise is **±0.007** to **0.009**. When the effect is smaller than the ruler, picking the best score is picking luck.

Seeded knowledge made the search faster, not truer; richer feedback changed nothing that transferred. What helped was boring: a larger evaluation fold halved the noise and tripled the surviving gain.

Campaign 2: aim where there is headroom

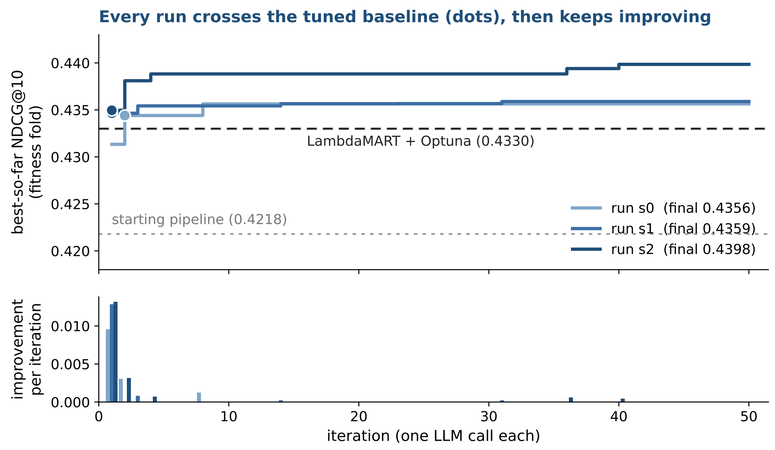
Same loop, wider space: features, models, losses, and ensembling behind leakage-safe helpers. This space passed the gate (**+0.013**, over **2.5×** the noise floor), so we ran it. Three runs, 50 iterations:

Method (fitness-fold trained)	Val NDCG@10	Test NDCG@10	Test book	Test revenue
Starting pipeline	0.4218	0.4222	0.4443	0.4105
LambdaMART + Optuna	0.4330	0.4296	0.4527	0.4231
Pipeline s0	0.4356	0.4316	0.4545	0.4257
Pipeline s1	0.4359	0.4322	0.4553	0.4215
Pipeline s2	0.4398	0.4352	0.4582	0.4239

Transfer audit on 59,902 held-out queries; bold = column best.

Every run beats the tuned baseline on held-out data; the best wins by **+0.0057** NDCG@10 (95% CI **[0.0041, 0.0073]**, $p < 0.0001$), with revenue matching.

Continual improvement, then proof



Best-so-far fitness NDCG@10 per iteration (top) and each iteration's increment (bottom). All three runs pass the tuned baseline within two iterations and keep finding increments to iteration 40; the table above shows what survived the audit.

It holds at full scale

280k train / 60k test queries	NDCG@10	NDCG@38
Tuned baseline (small-fold tuning)	0.4349	0.4983
Tuned baseline (full-scale tuning)	0.4544	0.5132
Pipeline s1 (small-fold tuning)	0.4490	0.5080
Pipeline s1 (baseline's tuned settings)	0.4619	0.5185

Features alone: 0.4587 (**+0.0043**); the ensemble adds **+0.0032**. Full-scale bootstrap: **+0.0075** NDCG@10, $p < 0.0001$.

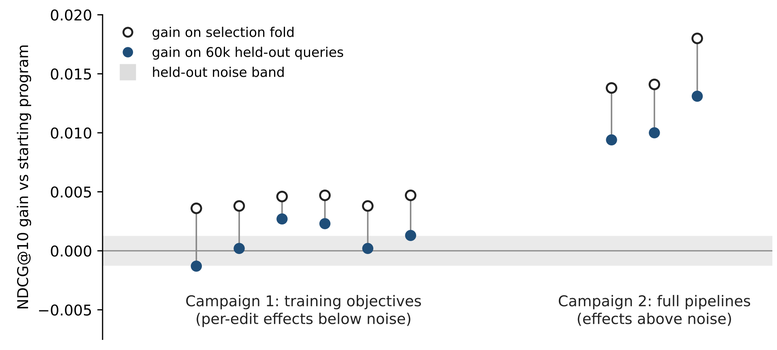
Kaggle late submission (one shot, hidden test set): discovered pipeline 0.5196 private NDCG@38 → would rank 20 of 340 (top 6%) in the 2013 challenge; tuned baseline 0.5140 (30th). Ten leaderboard places from the discovered structure alone.

What the search found

```
for c in [price, stars, review, loc1, loc2]:
  out[c+"_qrank"] = groupby(qid)[c].rank(pct=True)
out["prop_id_count"] = stats.count(df, "prop_id")
PIPELINE = {xgb:rank ndcg + lgbm:lambdarank + sklearn:ert,
  ensemble: zscore_weighted [0.5, 0.35, 0.15]}
```

The same readable core in all three runs: within-query percentile ranks, count encodings, a lean feature set, and a diverse ensemble under per-query z-scores: the ICDM 2013 winners' essential structure, rediscovered and extended autonomously.

Metrics after selection



The whole study in one picture: per-run NDCG@10 gain over the starting program, on the selection fold (open) versus 60k held-out queries (filled). Campaign 1's apparent gains collapse into the noise band after selection; campaign 2's survive.

Open everything: code, configs, seeds, logs, discovered programs, audit scripts at `github.com/shabazpatel/evorank`. Limitations: one dataset (the only public LTR set with conversion and revenue labels); three runs per condition; revenue is a proxy; audited for transfer, not online effects.

Use this on your ranking stack

1. Measure noise

Paired query bootstrap on the evaluation fold you can afford: that half-width is your noise floor.

2. Hand-build one candidate

One reference candidate from your domain's literature, before any LLM spend.

3. Gate the space

Require its gain ≥ **2.5×** the noise floor. No headroom, no loop: keep your budget.

4. Run guarded

Leakage-safe statistics, whitelists, budgets, and rejection messages the model learns from.

5. Audit, then believe

A held-out transfer audit, never the loop's own scores, decides what counts.

Contact: rayhanbp@umd.edu • shabaz.b.patel@gmail.com

Paper + proceedings: genai-ecommerce.github.io/GenAIECommerce2026